

Hardware-Aware CNN Optimization Toolkit: Automated Pruning, Convolution Transformation, with Heterogeneous Requests Scheduling on Intel Lunar Lake AI PC

Ritik Agarwal
MTech DSBA

Indian Institute of Science, Bengaluru
Email: ritikagarwal@iisc.ac.in

Abstract—This project proposes to efficiently use heterogeneous systems like Intel Lunar Lake AI PC by providing an automated hardware-aware optimization toolkit for deploying convolutional neural networks across CPU, GPU, and NPU. The toolkit takes a pretrained CNN as input and creates different variants of the CNN by applying structured filter pruning, optional dense to depthwise-separable-convolution conversion, and INT8 quantization. Each variant is thoroughly profiled across various parameters that will be discussed, before deploying the model on the device. The goal is to maximize performance and efficiency while maintaining a target accuracy. A Python implementation built on PyTorch, OpenVINO, and NNCF will be evaluated on multiple CNN architectures and optimization configurations, reporting speedup, energy, and accuracy trade-offs under different deployment strategies.

Index Terms—Convolutional neural networks, model compression, pruning, quantization, heterogeneous computing, Intel Lunar Lake, OpenVINO, NNCF.

I. INTRODUCTION

Deep convolutional neural networks (CNNs) are widely used in computer vision and classification workloads, but their computational and memory demands pose a challenge for energy and latency constrained systems. Early CNNs like LeNet-5 [1], which was built using just 5 layers and only 2.6K weights, do not require that much compute. However, the more recent ones, such as GoogleNet [2] with 6M weights or ResNet-50 [3], which exceeds 23M parameters, are much more compute-hungry. To maintain the cost-effectiveness of such neural networks, they need to be efficiently deployed and managed across both hardware and software levels.

Modern client and edge platforms, such as Intel Lunar Lake, provide heterogeneous compute resources combining general-purpose CPU cores, integrated GPUs, and dedicated NPUs for AI acceleration. CPUs are designed for general-purpose workloads, whereas GPUs offer a high degree of parallelism but at the cost of higher power consumption. NPUs can execute a high number of operations simultaneously [4], while consuming much lower power than GPUs.

However, mapping an unoptimized CNN directly onto this heterogeneous hardware often underutilizes the compute

available. Deploying a CNN completely on a GPU incurs high power consumption, while a CPU deployment drastically reduces performance. Most neural networks can also benefit from certain techniques, using which different layers can leverage the heterogeneous compute on the device to provide the most optimal throughput and power efficiency.

This project targets the problem of automatically transforming and scheduling CNNs to exploit Lunar Lake’s CPU–GPU–NPU ensemble. Different CNNs will be studied and profiled based on the number of weights, MAC operations, and power consumption to understand which techniques can provide the most benefit for different hardware options. The focus is on building a practical end-to-end Python toolkit that can be applied to arbitrary CNN models, rather than on proposing a single new compression algorithm.

In this Phase 2 report, we present the preliminary benchmark results from analysing LeNet-5, and AlexNet. We also finalize our approach for heterogeneous deployment of the CNNs to maximize device utilization. We conclude with our final goal to be achieved by the end of the project.

II. RELATED WORK

Research on CNN model compression has explored pruning, quantization [5], [6], and architecture transformations to reduce computation and memory footprint while preserving accuracy. Deep work has been done on structured pruning [7] to solve these challenges further.

Quantization techniques, including post-training quantization [8] and quantization-aware training [9], enable deployment of CNNs in reduced precision (e.g., INT8) for improved throughput and energy efficiency on accelerators and CPUs, while maintaining target accuracy. However, not all precisions are supported across all devices. In this project, we split the layers onto compatible devices while considering this factor.

Depthwise separable convolutions [10], popularized by families such as MobileNet and Xception, restructure convolution layers to lower MAC counts and parameter counts. Transforming a dense layer to depthwise separable theoretically reduces the number of parameters, but it can also end up

under-utilizing the memory bandwidth [11]. We analyze which layers can benefit in terms of performance from dense-to-DSC conversion, while hitting accuracy targets.

Hardware-aware optimization frameworks and compilers have emerged to automatically tailor neural networks to target platforms. We use OpenVINO on Intel AIPC, which integrates quantization, pruning, and heterogeneous deployment [12] of models across devices. We test with different configurations of the neural network and load balance requests across all available devices to maximize throughput.

III. METHODOLOGY

In Phase 2 of this project, we analyzed two Convolutional Neural Networks (CNNs): LeNet-5 and AlexNet. To compare performance, we benchmarked these models across various batch sizes and devices, including CPUs, GPUs, and NPUs. Our primary target metric is throughput, measured as the number of frames inferred per second (FPS).

A. Class Based CNN Implementation

To maintain extensibility in the project, a class structure has been created, compatible with any CNN, to integrate it into the application flow. This class serves the following key functions:

- **Model Initialization:** The constructor initializes the CNN and stores the OpenVINO Intermediate Representation (IR).
- **Model Analytics:** It extracts layer-level architectural metrics, such as MAC (Multiply-Accumulate) operation counts, estimated memory traffic, arithmetic intensity, and supported devices. These metrics help determine whether a layer is compute-bound or memory-bound, which guides the degree of pruning required.
- **Pre-Processing:** It provides a custom data pre-processing function for inputs during benchmarking.
- **Prediction:** It loads the model onto a target device, processes a batch of prepared images, and runs inference. The inference logic can be fully customized as needed.
- **Setup Training and Datasets:** It configures the training environment by defining the appropriate dataset, loss function, and optimizer for the specific model.
- **Finetune Epoch:** This method directly integrates with the training loop to fine-tune the model during pruning and quantization. This allows for seamless experimentation with different modification strategies.

By implementing these required functions, any CNN can utilize this class and execute the toolkit scripts. The toolkit's ultimate goal is to produce a modified model architecture—through pruning and quantization—that maximizes throughput while keeping accuracy within 3% of the original model.

B. Benchmark Harness

We developed a novel benchmarking harness to process the model classes described above. This harness tests various combinations of models, batch sizes, and benchmarking methods to generate the following metrics:

- Throughput (FPS).

- Latency percentiles for a single request.
- Layer-level latencies during inference.

The harness evaluates two methods of benchmarking:

- **Real-Data:** This method uses a real-world dataset, such as MNIST, to create image batches based on the configuration. The images are pre-processed using the CNN class method, the model is compiled for the target device, and the batches are passed through the prediction function. Recording the latency and throughput during this process provides a realistic estimate of end-to-end performance.
- **OpenVINO Benchmark App:** To gather more fine-grained metrics, we utilize OpenVINO's built-in benchmarking tool, enhanced to record layer-level execution times. Similar to the Real-Data method, inputs are generated based on the configured batch size, and the model is compiled for the specific hardware.

We also created an additional script to generate comparison charts to get a view of the performance data. This helps us understand the model to device compatibility and the performance limitation of each device.

C. Experimental Training Loop

A central goal of this project is to create model architectures that minimize computation while preserving accuracy. To ensure this process is reliable and reproducible, we are developing a Python script to orchestrate the training loop. This orchestrator automatically applies various quantization, pruning, and modification strategies to the models. It leverages the CNN class methods for training and evaluation, guaranteeing that every model undergoes a consistent training cycle.

To accelerate training, we apply an approach similar to gradient descent. We begin with a high pruning ratio and take large steps to reduce it. As the model reaches its target accuracy, we gradually decrease the pruning gap between iterations. This step-down logic is also applied to the number of epochs required per iteration. Ultimately, this process yields a modified model exported in PyTorch format. This final model is then benchmarked using the exact same parameters as the original, to keep a fair, apples-to-apples comparison.

IV. RESULTS

In this section, we present the benchmarking results for LeNet-5 and AlexNet across different hardware accelerators (CPU, GPU, and NPU) and varying batch sizes. Our analysis starts from high-level throughput metrics all the way to granular, layer-wise performance characteristics.

A. Computational Complexity

To get an idea of the performance variations across models, we first look at the baseline computational requirements. Table I details the total Multiply-Accumulate (MAC) operations for both models across different batch sizes. As expected, AlexNet requires a significantly higher number of computational operations compared to LeNet-5, which directly influences how effectively it can utilize hardware accelerators.

TABLE I
TOTAL MAC OPERATIONS BY BATCH SIZE

Model	bs=100 (GMACs)	bs=500 (GMACs)
AlexNet	71.42	357.09
LeNet	0.04	0.21

B. Throughput Analysis

To evaluate hardware utilization, we calculated the normalized throughput, using the CPU’s performance as the baseline (1.00X) for each model and batch size configuration. Figure 1 illustrates the performance gains provided by the GPU and NPU over the CPU.

A clear shift in hardware efficiency is observed between the two models. For LeNet-5, the performance gains on the GPU are marginal, and the NPU actively underperforms compared to the CPU. This behavior indicates that for small models with low MAC counts, the data transfer and initialization overheads hide the benefits of parallel compute. Conversely, AlexNet, with its high computational density, demonstrates significant throughput gains on both the GPU and NPU, showcasing their capability to accelerate heavily compute-bound workloads.

Normalized Throughput Comparison Across Devices

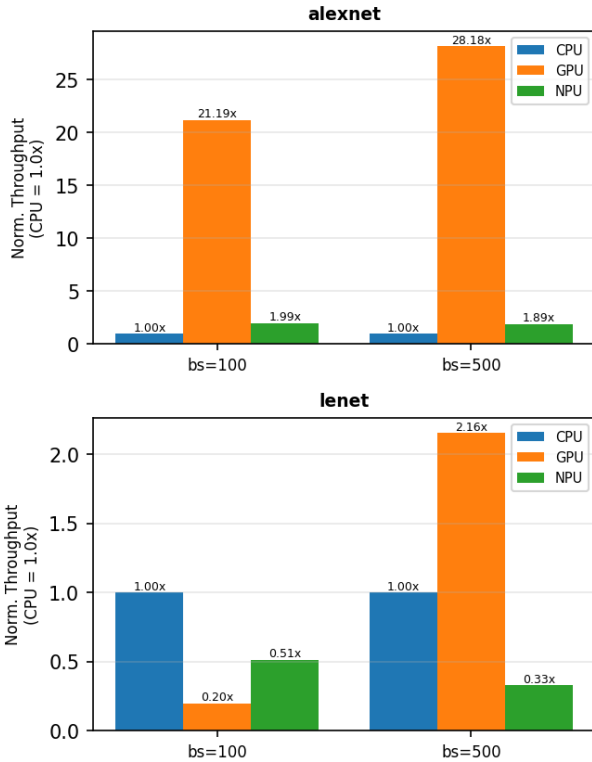


Fig. 1. Normalized Throughput (CPU = 1.00X) vs Model and Batch Size across CPU, GPU, and NPU.

C. Layer-Level Performance and Profiling

To understand the macro-level throughput behavior, we analyzed the models at the layer level.

1) *Architectural Breakdown:* Table II presents a side-by-side architectural profile for both models, detailing layer types, MACs, Memory Traffic, Arithmetic Intensity, and supported devices. This profile helps identify which layers act as memory bottlenecks versus compute bottlenecks.

TABLE II
PER-LAYER ARCHITECTURAL PROFILE (BATCH SIZE = 100)

(a) AlexNet					
Layer	Type	MACs (ops)	Mem (B)	Arith Int. (ops/B)	Devices
Conv1	Conv	7,027,680,000	137,744,128	51.02	CPU, GPU, NPU
Conv2	Conv	22,394,880,000	75,878,400	295.14	CPU, GPU, NPU
Conv3	Conv	11,214,028,800	41,591,808	269.62	CPU, GPU, NPU
Conv4	Conv	14,952,038,400	46,802,944	319.47	CPU, GPU, NPU
Conv5	Conv	9,968,025,600	36,970,496	269.62	CPU, GPU, NPU
FC1	MatMul	3,774,873,600	156,319,744	24.15	CPU, GPU, NPU
FC2	MatMul	1,677,721,600	70,385,664	23.84	CPU, GPU, NPU
FC3	MatMul	409,600,000	18,422,400	22.23	CPU, GPU, NPU
Total		71,418,848,000			

(b) LeNet-5					
Layer	Type	MACs (ops)	Mem (B)	Arith Int. (ops/B)	Devices
Conv1	Conv	11,760,000	2,291,800	5.13	CPU, GPU, NPU
Conv2	Conv	24,000,000	1,120,000	21.43	CPU, GPU, NPU
FC1	MatMul	4,800,000	400,000	12.00	CPU, GPU, NPU
FC2	MatMul	1,008,000	121,920	8.27	CPU, GPU, NPU
FC3	MatMul	84,000	40,960	2.05	CPU, GPU, NPU
Total		41,652,000			

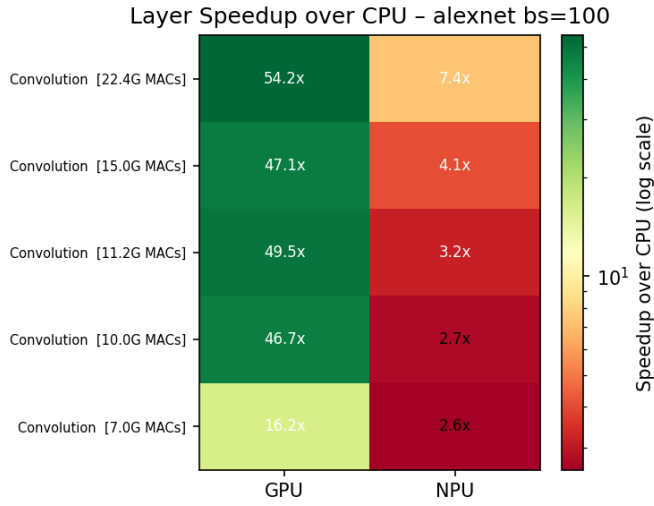
2) *Compute-Bound Layer Speed-up:* Focusing on the compute-bound layers (Convolution and MatMul/Fully Connected), we generated a speed-up heatmap (Figure 2) to compare GPU and NPU execution times against the CPU.

The heatmap reveals a strong positive correlation between a layer’s MAC operations and its relative speed-up on accelerators. Layers with higher MAC counts (prominent in AlexNet) exhibit significant performance gains on the GPU and NPU due to effective parallelization. In contrast, layers with lower MAC counts (such as those in LeNet-5) fail to saturate the compute units of the accelerators, resulting in diminished returns or even performance degradation compared to the CPU.

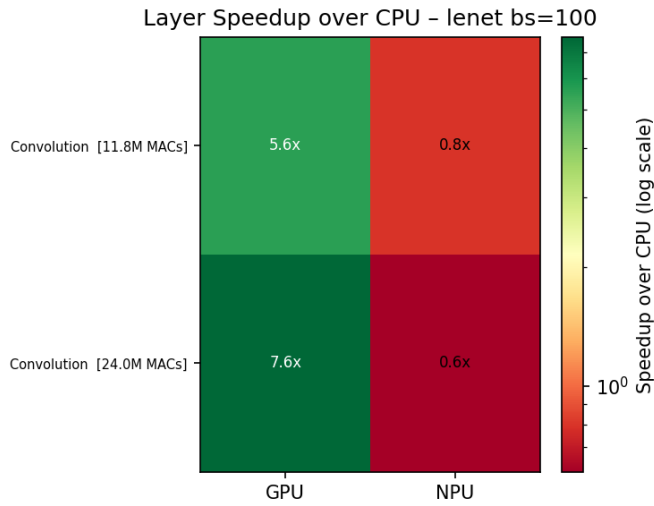
V. CONCLUSION

The experiments conducted in Phase 2 point to the fact that each hardware device has specific limitations, and their overall utilization improves significantly with more compute-bound models. We also observed that splitting a single model’s layers across multiple devices incurs a high data movement overhead, which severely decreases throughput.

As a result, we have adjusted our execution strategy. To achieve our original goal of high hardware utilization, we will now focus on scheduling independent inference requests across all available hardware devices simultaneously, rather than splitting individual model layers across them. Additionally, quantization, pruning, and model architecture modifications remain central to the project, ensuring we maximize throughput while maintaining accuracy.



(a) AlexNet Heatmap - Batch Size 100



(b) LeNet-5 Heatmap - Batch Size 100

Fig. 2. Layer Speed-up Heatmap (CPU vs. GPU/NPU) for Convolution and MatMul layers, annotated with corresponding MAC counts. (a) AlexNet demonstrates significant acceleration on high-MAC layers. (b) LeNet-5 shows diminished returns on low-MAC layers.

Moving forward, we plan to include roofline plots into our analysis to better visualize how efficiently each hardware device is being utilized relative to its theoretical performance limits. By the end of the project, we aim to include more complex CNNs and clearly show the overall throughput gains achieved through the combination of concurrent hardware scheduling and targeted model optimizations.

REFERENCES

- [1] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [2] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. E. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with

- convolutions," *CoRR*, vol. abs/1409.4842, 2014. [Online]. Available: <http://arxiv.org/abs/1409.4842>
- [3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," 2015. [Online]. Available: <https://arxiv.org/abs/1512.03385>
- [4] T. Tan and G. Cao, "Efficient execution of deep neural networks on mobile devices with npu," in *Proceedings of the 20th International Conference on Information Processing in Sensor Networks (Co-Located with CPS-IoT Week 2021)*, ser. IPSN '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 283–298. [Online]. Available: <https://doi.org/10.1145/3412382.3458272>
- [5] A. Kuzmin, M. Nagel, M. van Baalen, A. Behboodi, and T. Blankevoort, "Pruning vs quantization: Which is better?" in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 62 414–62 427. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/c48bc80aa5d3cbbdd712d1cc107b8319-Paper-Conference.pdf
- [6] Q. Qi, Y. Lu, J. Li, J. Wang, H. Sun, and J. Liao, "Learning low resource consumption cnn through pruning and quantization," *IEEE Transactions on Emerging Topics in Computing*, vol. 10, no. 2, pp. 886–903, 2022.
- [7] Y. He and L. Xiao, "Structured pruning for deep convolutional neural networks: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 5, pp. 2900–2919, 2024.
- [8] P. Kluska and M. Zieba, "Post-training quantization methods for deep learning models," in *Intelligent Information and Database Systems*, N. T. Nguyen, K. Jearanaitanakij, A. Selamat, B. Trawiński, and S. Chittaya-sothorn, Eds. Cham: Springer International Publishing, 2020, pp. 467–479.
- [9] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, "A white paper on neural network quantization," 2021. [Online]. Available: <https://arxiv.org/abs/2106.08295>
- [10] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," 2017. [Online]. Available: <https://arxiv.org/abs/1610.02357>
- [11] R. Shang, J. He, J. Wang, K. Xu, L. Jiao, and R. Stolkin, "Dense connection and depthwise separable convolution based cnn for polarimetric sar image classification," *Knowledge-Based Systems*, vol. 194, p. 105542, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0950705120300411>
- [12] F. Z. Guerrouj, M. ABOUZAHER, M. RAMZI, and E. M. ABDALI, "Analysis of the acceleration of deep learning inference models on a heterogeneous architecture based on openvino," in *2021 4th International Symposium on Advanced Electrical and Communication Technologies (ISAECT)*, 2021, pp. 01–05.